

PROJECT 03 · WEB APP SECURITY

VULNERABILITY ASSESSMENT & PROFESSIONAL REPORTING

Report By: Shravani Joshi

Vulnerability Assessed: Cross-Site Request Forgery (CSRF)

Assessment Platform: PortSwigger Web Security Academy

Lab: CSRF vulnerability with no defenses

Executive Summary

This report presents the findings of a Web Application Security Assessment conducted on an intentionally vulnerable application provided through PortSwigger Web Security Academy. The objective of the assessment was to identify and safely demonstrate a Cross-Site Request Forgery (CSRF) vulnerability within the authorized laboratory environment.

The assessment focused on the application's email update functionality available through the authenticated user's account. The functionality was found to accept a state-changing POST request without requiring a CSRF token or an equivalent mechanism to verify that the request was intentionally initiated by the authenticated user.

A crafted HTML form was created on the provided exploit server to submit a forged request to the vulnerable email-change endpoint. The request was configured to automatically submit when the exploit page was loaded. After delivering the exploit to the simulated victim, the victim's email address was successfully changed to the attacker-controlled value.

The PortSwigger Web Security Academy environment subsequently confirmed successful exploitation by marking the lab as solved. The assessment demonstrates that the absence of CSRF protection can allow unauthorized state-changing actions to be performed through a victim's authenticated session.

Objective

The objective of this assessment was to:

- Identify a CSRF vulnerability in an intentionally vulnerable web application.
- Understand the cause and behaviour of the vulnerability.
- Safely demonstrate exploitation within the provided laboratory environment.
- Document the exploitation process using screenshots.
- Analyze the potential impact of the vulnerability.
- Provide professional remediation recommendations.

Scope

The assessment was restricted to the intentionally vulnerable application and exploit server provided by PortSwigger Web Security Academy.

- Lab: CSRF vulnerability with no defenses
- Target Functionality: Email address update
- Affected Endpoint: /my-account/change-email

No real-world websites, systems, or third-party applications were targeted during the assessment.

Vulnerability Overview

Vulnerability: Cross-Site Request Forgery (CSRF)

Severity: Medium

Status: Successfully Exploited

Affected Functionality: Change Email

What is CSRF?

Cross-Site Request Forgery is a web application vulnerability that allows an attacker to cause an authenticated user's browser to perform an unintended action on a trusted website. The attack is possible when an application relies on the user's existing authenticated session but does not use an additional mechanism, such as a CSRF token, to verify that the request was intentionally generated by the user.

Exploitation Procedure

1. Initial Application Access

The lab provides an intentionally vulnerable web application along with an exploit server that can be used to host and deliver a crafted CSRF payload.



Figure 1: Homepage of the vulnerable application

The vulnerable application was accessed through the provided lab environment. The application initially displayed its homepage and provided navigation to the login functionality. The credentials supplied by the laboratory were used to authenticate to the application. After successful authentication, the application provided access to the user's account area.

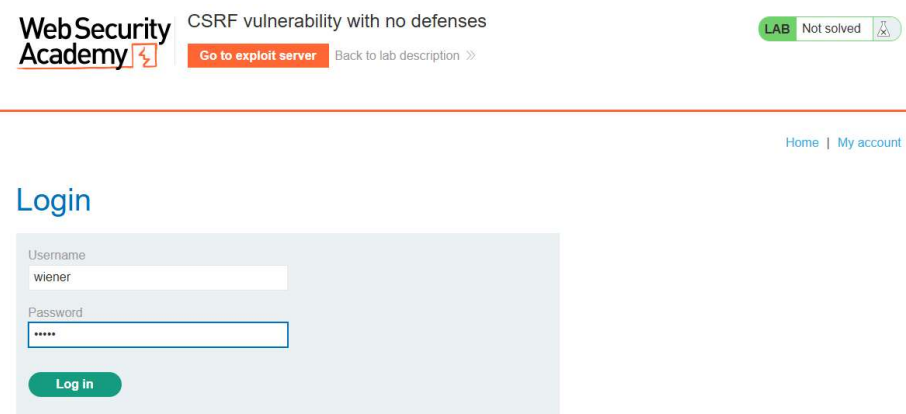


Figure 2: Login page of the vulnerable application

2. Identification of Vulnerable Functionality

After logging in, the My Account page was accessed. The page displayed the authenticated user's username and email address and provided an Update email functionality.

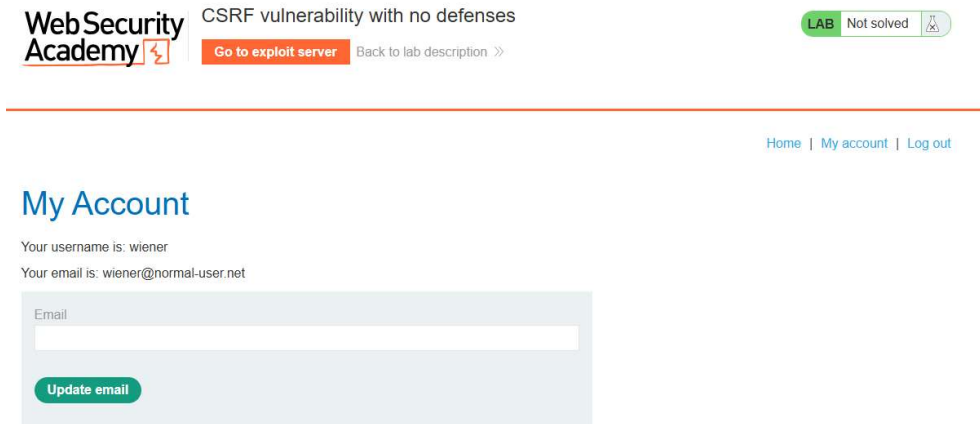


Figure 3: My Account page containing the email update functionality

The functionality was identified as a suitable CSRF target because it performs a state-changing operation on the authenticated user's account. The legitimate functionality accepts an email parameter and sends the update request to: `/my-account/change-email`

3. Vulnerability Analysis

The vulnerability occurs because the email update request does not contain a CSRF token or another mechanism that allows the server to verify that the request was intentionally generated by the legitimate application.

The application therefore relies on the authenticated user's session when processing the request. An attacker can take advantage of this behaviour by creating a malicious webpage containing a form that submits a request to the vulnerable endpoint.

When an authenticated victim accesses the malicious page, the browser can submit the forged request using the victim's existing authenticated session. The application then processes the request as a legitimate authenticated operation.

The absence of CSRF protection is therefore the primary root cause of the identified vulnerability.

4. Proof of Concept

Exploit Crafting:

The provided PortSwigger exploit server was accessed to create a malicious HTML page.

The following payload was used:

```
<form method="POST" action="https://<LAB-ID>.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="anything@web-security-academy.net">
</form>
<script>
  document.forms[0].submit();
</script>
```

The laboratory-specific ID was inserted into the target URL before the exploit was stored.

This is your server. You can use the form below to save an exploit, and send it to the victim.
Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0ae5001d045e3c64810cd83d01dc00f5.exploit-server.net/exploit>

HTTPS

☒

File:

Head:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

Body:

```
<form method="POST" action="https://0abd001204843c898116d9980039007a.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="anything@web-security-academy.net">
</form>
<script>
  document.forms[0].submit();
</script>
```

[Store](#) [View exploit](#) [Deliver exploit to victim](#) [Access log](#)

Figure 4: Crafted CSRF payload on the exploit server

Exploit Explanation:

The payload contains a hidden HTML form that targets the vulnerable email-change endpoint. The method="POST" attribute specifies that the request should use the POST method expected by the application.

The action attribute identifies the vulnerable endpoint:

/my-account/change-email

The hidden email field supplies the email address that the attacker wants the application to process.

The JavaScript statement: document.forms[0].submit();

automatically submits the form when the page is loaded. This means that the victim does not need to manually click the Update email button.

Most importantly, the request does not contain a CSRF token because the vulnerable application does not require one.

Exploit Storage and Delivery:

After configuring the payload, the exploit was stored on the provided exploit server. The exploit was then viewed to verify that the crafted page was available.

The Deliver exploit to victim functionality was subsequently used to simulate delivery of the malicious page to the authenticated victim. This reproduced the scenario in which an attacker attempts to cause an authenticated user to perform an unintended account action.

5. Expected Result

A properly protected application should reject the forged request because it does not contain a valid CSRF token or equivalent request validation mechanism.

The victim's email address should remain unchanged.

6. Actual Result

The forged request was successfully accepted by the application.

After the exploit was delivered, the authenticated user's email address changed from: `wiener@normal-user.net`

to: anything@web-security-academy.net

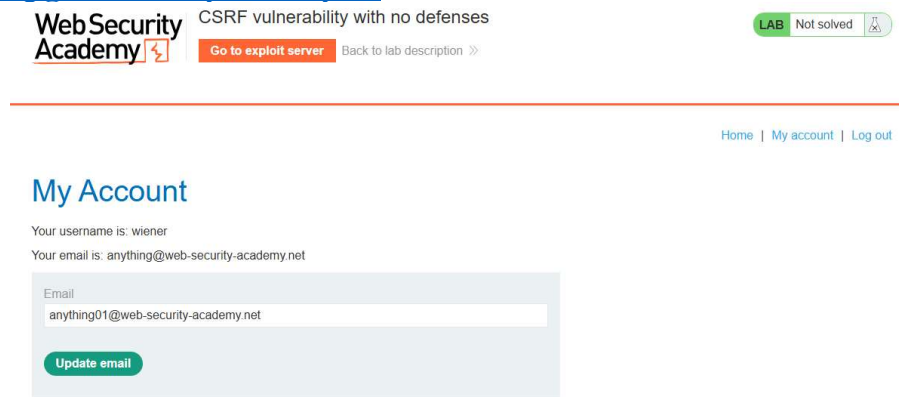


Figure 5: Email address changed after successful CSRF exploitation

The user did not manually submit the legitimate email update form to perform this change. This confirms that the application processed the forged request as an authenticated state-changing operation.

7. Successful Exploitation

The PortSwigger Web Security Academy subsequently displayed: “Congratulations, you solved the lab!”

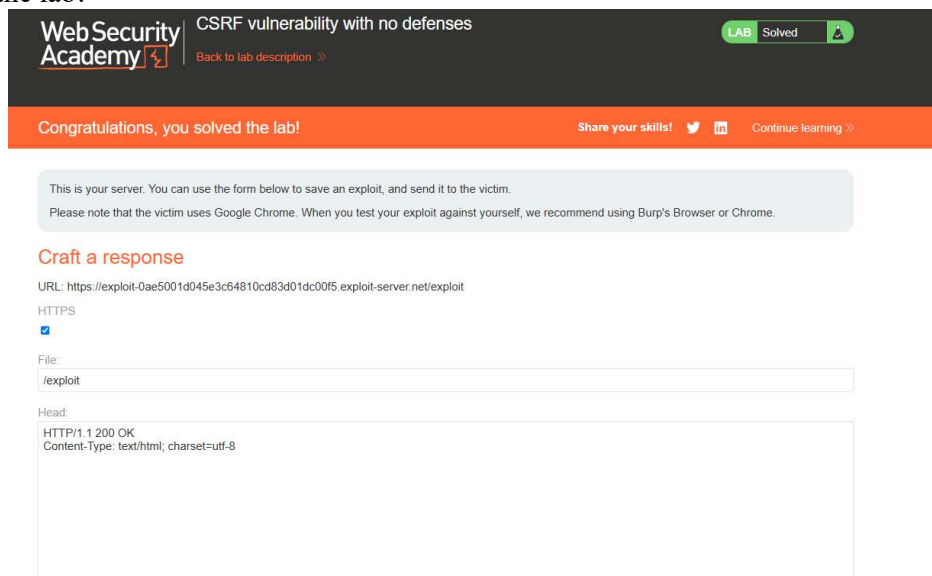


Figure 6: Successful exploitation complete

The solved status confirms that the CSRF attack met the laboratory's successful exploitation criteria.

8. Access Log Evidence

The exploit server access logs were reviewed following the attack. The logs recorded activity associated with accessing and delivering the hosted exploit, including requests to the exploit resource and POST requests generated during the testing process.



Figure 7: Exploit server access logs

Impact

The identified CSRF vulnerability allows an attacker to cause an authenticated user to perform an unintended state-changing action.

In the demonstrated scenario, the attack successfully modified the victim's email address. Similar CSRF weaknesses affecting other state-changing functionality could potentially allow unauthorized actions such as:

- Modification of account information.
- Changes to user preferences.
- Modification of security settings.
- Changes to notification or communication settings.
- Other actions available to the authenticated user.

The potential severity would increase if sensitive functionality such as password changes, account recovery settings, financial operations, or administrative actions were similarly vulnerable.

Risk Assessment

Severity: Medium

The vulnerability was successfully exploited and resulted in unauthorized modification of account information.

The demonstrated impact within the laboratory is limited to the email-change functionality. However, in a production application, the risk could be considerably higher if similar CSRF weaknesses were present in more sensitive functionality.

The overall risk should therefore be evaluated across all authenticated state-changing endpoints.

Remediation

• Implement CSRF Tokens

The primary recommendation is to implement strong, unpredictable CSRF tokens for all security-sensitive state-changing requests.

A token should be generated by the server and associated with the user's session. The token should be included in legitimate forms and validated by the server before processing the requested action.

Requests containing a missing, invalid, or incorrect token should be rejected.

- **Perform Server-Side Validation**
CSRF protection should be enforced on the server side rather than relying solely on client-side controls.
The server should verify the validity of the CSRF token before performing any state-changing operation.
- **Configure SameSite Cookies**
Authentication cookies should use an appropriate SameSite attribute, such as Lax or Strict where compatible with the application's requirements.
This provides an additional layer of protection by restricting when cookies are sent with cross-site requests.
- **Validate Request Origin**
For sensitive operations, the application can additionally validate the Origin header and, where appropriate, the Referer header.
These controls should be treated as defense-in-depth and should not replace proper CSRF token validation.
- **Review All State-Changing Functions**
CSRF protection should be implemented consistently across the application.
Other authenticated operations such as profile updates, password changes, security settings, administrative actions, and other account-management functions should be reviewed for the same weakness.

Retesting Recommendations

After remediation, the following tests should be performed:

- Verify that legitimate email update requests containing a valid CSRF token are accepted.
- Verify that requests without a CSRF token are rejected.
- Verify that requests containing an invalid CSRF token are rejected.
- Verify that a token from another user session cannot be reused.
- Verify that appropriate SameSite cookie settings are enabled.
- Review other authenticated state-changing endpoints for equivalent protection.

Evidence Summary

Evidence	Description
Figure 1	Homepage of the vulnerable application
Figure 2	Login page of the vulnerable application
Figure 3	My Account page containing the email update functionality
Figure 4	Crafted CSRF payload on the exploit server
Figure 5	Email address changed after successful CSRF exploitation
Figure 6	Successful exploitation complete
Figure 7	Exploit server access logs

Findings Summary

Field	Details
Finding	Cross-Site Request Forgery (CSRF)
Severity	Medium
Lab Name	CSRF vulnerability with no defenses - PortSwigger Web Security Academy
Status	Successfully Exploited
Affected Functionality	Email Address Update

Affected Endpoint	/my-account/change-email
Authentication Required	Yes
Root Cause	Absence of CSRF token or equivalent request validation mechanism
Attack Method	Forged POST request submitted through a HTML form
Impact	Unauthorized modification of the authenticated user's email address
Recommended Fix	Implement server-side CSRF tokens, configure SameSite cookies, validate request origin, and protect all authenticated state-changing endpoints

Conclusion

The assessment successfully identified and demonstrated a Cross-Site Request Forgery vulnerability in the email update functionality of the intentionally vulnerable PortSwigger Web Security Academy application.

The vulnerability was caused by the absence of CSRF protection on a state-changing request. A crafted HTML form was successfully used to generate a forged request to the vulnerable endpoint. After the exploit was delivered, the authenticated user's email address was changed to an attacker-controlled value without requiring the user to intentionally submit the legitimate update form.

The successful exploitation was confirmed by the PortSwigger laboratory environment. The vulnerability can be mitigated through the implementation of server-side CSRF tokens, appropriate cookie security controls, request-origin validation, and consistent protection of all authenticated state-changing operations.